

AIMLCZG546 · SESSION 1 · COMPANION READER

Foundations of ML Systems Engineering

SDLC, Data Science, ML Pipeline, Three Generations, Domains and LLMs

Session 1 · Read alongside the lecture slides · Every concept explained from scratch

How to use this companion. Five colour-coded boxes appear throughout. **Blue** boxes give the core intuition in plain words. **Amber** boxes anchor the idea to an everyday situation you already understand. **Green** boxes walk through a concrete example step by step. **Red** boxes flag common pitfalls. **Purple** boxes state the single most important line to remember.

1 Software Development Life Cycle (SDLC)

Every software product you use was built using a repeatable process. Think of a banking app, a ride-hailing service, or a hospital record system. That process is the **Software Development Life Cycle (SDLC)**. It is a structured sequence of phases. It guides a team from an idea to a shipped product. Then it keeps that product running.

The intuition

The SDLC is a recipe. A recipe tells a chef what to do, in what order, and when each step is done. The SDLC does the same for a software team. First, understand what to build. Then design it. Then write the code. Then test it. Then ship it. Then maintain it.

★ Everyday picture

Think of building a house. Before a single brick is laid, the owner explains their needs (requirements). An architect draws blueprints (design). Builders construct the walls (implementation). Inspectors check for cracks (testing). The family moves in (deployment). A plumber fixes leaks over the years (maintenance). A software team follows the same six steps in the same order.

The six classic SDLC phases are:

1. **Planning** — what are we building, and is it feasible?
2. **Requirements Analysis** — what exactly must the system do?
3. **System Design** — how will it be structured (architecture, database, APIs)?
4. **Implementation (Coding)** — write the actual code.
5. **Testing & Quality Assurance** — does it work correctly and safely?
6. **Deployment & Maintenance** — release it and keep it healthy.

1.1 Roles in a Software Team

A software project involves many specialists, each responsible for a different slice of the process.

Role	Phase focus	Core responsibility
Business Analyst	Requirements	Translates business needs into technical specifications
Project Manager	All phases	Plans timelines, manages risks, tracks progress
Product Owner	Requirements + Delivery	Represents users; prioritises features
Software Architect	Design	Designs high-level structure and technology choices
UX/UI Designer	Design + Testing	Creates user interfaces and experience flows
Developer	Implementation	Writes and reviews code
QA / Tester	Testing	Writes and runs tests; reports defects
Scrum Master	All (in Agile)	Removes blockers; facilitates Agile ceremonies
Team Lead	Implementation	Guides the developer team; does code review

Worked example: Who does what on a food-delivery app

Imagine building a Swiggy-like app. Each role owns a slice of the work.

- Step 1. Business Analyst.** Interviews restaurant owners and customers. Writes a requirements document.
- Step 2. Software Architect.** Decides to use microservices: separate services for orders, payments, and delivery.
- Step 3. UX Designer.** Draws the app screens and the user flow.
- Step 4. Developers.** Build each service and write the code.
- Step 5. QA Engineers.** Test that an order deducts the right amount and notifies the right partner.
- Step 6. Project Manager.** Ensures the first release ships before the next festive season.
- Step 7. DevOps Engineer.** Deploys the app to the cloud and sets up monitoring.

1.2 Evolution to Cloud-Native

Traditional SDLC produced software released once every six to twelve months. Modern software teams ship multiple times per day. The shift happened through three combined innovations:

- **Agile** — work in short sprints (1–3 weeks); deliver working software continuously.
- **DevOps** — development and operations teams share responsibility. Testing and deployment are automated.
- **Cloud Native** — use containers (Docker), orchestration (Kubernetes), and microservices. Systems become small, independent, and always running.

Where this shows up in ML/AI. Every ML team runs inside some form of SDLC. ML adds new phases: data collection, model training, and evaluation. These do not appear in traditional SDLC. This course shows how to adapt the SDLC to handle them.

Key takeaway

The SDLC is the backbone of every software project. It doesn't change when ML enters the picture — ML adds new phases inside it.

SDLC: six phases from idea to running system



Figure 1: The six SDLC phases in order. Each phase feeds the next. Read from left to right.

2 Data Science and the AI/ML Landscape

Data Science is the study of data: collecting it, cleaning it, analysing it, and drawing actionable insights from it. The goal is to extract value that would otherwise remain hidden inside raw numbers.

The intuition

Imagine a mountain of sand. Data Science is the sieve that separates the gold flakes (insights) from the rest (noise). The sieve has five layers: collect the sand, clean it, analyse it, build a prediction model, then show the results.

★ Everyday picture

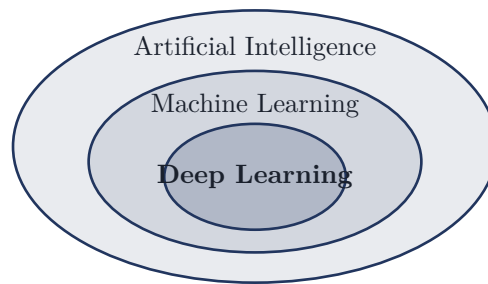
A doctor reviews thousands of patient records. They notice: “patients with these three symptoms are 80 % more likely to develop diabetes.” That is data science. They collected data (records), cleaned it (removed duplicates), and spotted the pattern. They built a mental prediction model. Then they shared the finding — the 80 % figure — with colleagues.

The standard Data Science workflow:

1. **Data Collection** — gather raw data from databases, APIs, sensors, surveys.
2. **Data Preprocessing** — fix missing values, remove duplicates, normalise formats.
3. **Exploratory Data Analysis (EDA)** — visualise distributions, spot correlations, understand the data.
4. **Modelling** — train a statistical or ML model on the cleaned data.
5. **Deployment & Visualisation** — share the predictions or dashboards with decision-makers.

2.1 Where AI, ML, and Data Science Overlap

These three terms are often confused. Here is the precise relationship:



- **Artificial Intelligence (AI)** — the broad goal of making machines behave intelligently.
- **Machine Learning (ML)** — a subset of AI where machines learn from data rather than being hand-programmed with rules.
- **Deep Learning** — a subset of ML that uses multi-layer neural networks; excels when data is large and unstructured (images, text, audio).
- **Data Science** — overlaps all three; focuses on extracting insights from data, often using ML as a tool.

Where this shows up in ML/AI. Data Science is often the starting point of an ML project. Before a model can be trained, a data scientist must collect and clean the data. The roles overlap significantly; in small companies one person may do all of them.

3 Machine Learning Basics

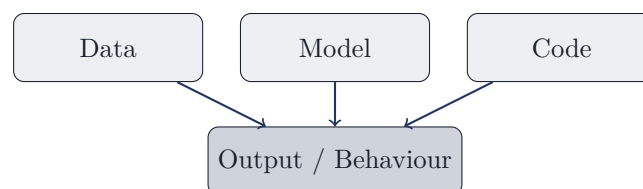
Machine Learning is a branch of AI. A program improves at a task through experience (data). You do not need to hand-code the rules. The program finds them itself.

The intuition

Traditional programming is like giving someone a recipe: “If the tomato is red and soft, it is ripe.” ML is like showing that person a thousand tomatoes labelled “ripe” or “not ripe” and letting them figure out the rule themselves. The rule they learn may be far more nuanced than anything you could write down.

3.1 The Three-Variable Complexity of ML

Traditional software has one moving part: the *code*. You change the code, the behaviour changes. ML systems have three moving parts that all affect the output:



Change the data (add more examples or remove biased ones) and the behaviour changes. Change the model architecture (switch from logistic regression to a neural network) and the behaviour changes. Change the code (bug in the preprocessing pipeline) and the behaviour changes. This three-way dependency makes debugging ML systems much harder than debugging traditional software.

⚠ Watch out

Do not test an ML system by only testing the code. A bug in the training data (mislabelled examples, wrong units) will silently produce wrong predictions even when all the code is correct. You must test data quality, model behaviour, and code together.

3.2 The Model Lifecycle

A **model** is a trained algorithm that accepts inputs and produces predictions or decisions. A model is not static — it has its own lifecycle:

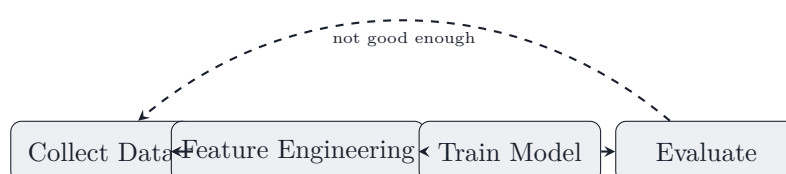
1. **Development** — choose an algorithm, train it on labelled data, tune hyperparameters.
2. **Evaluation** — measure accuracy, precision, recall, AUC on a held-out test set.
3. **Versioning** — save the trained model file and all metadata (training date, dataset version, metrics).
4. **Deployment** — serve the model via an API so applications can call it.
5. **Monitoring** — track prediction quality in production; detect data drift and performance degradation.
6. **Retraining** — retrain on fresh data when performance drops below an acceptable threshold.

Where this shows up in ML/AI. Every MLOps tool (MLflow, SageMaker, Vertex AI) is built around this lifecycle. When you hear “ML pipeline” in industry, this is what people mean.

4 The ML Pipeline

The **ML pipeline** is the end-to-end sequence of steps that takes raw data and produces a deployed, monitored model. It has two linked loops.

4.1 The Static (Training) Pipeline



4.2 The Production Pipeline

After the model passes evaluation, it enters production:

**🔗 Worked example: Building a spam filter end to end**

We build a spam filter using the full ML pipeline. Start with 100,000 labelled emails.

Step 1. Collect. Gather 100,000 emails each labelled “spam” or “not spam”.

Step 2. Features. Extract word counts, sender domain reputation, and link count from each email.

Step 3. Train. Fit a logistic regression model on 80,000 training emails.

Step 4. Evaluate. Test on 20,000 held-out emails. Result: precision 94 %, recall 91 %.

Step 5. Deploy. Expose the model as a REST API. Email servers call it for each incoming message.

Step 6. Monitor. Track how often users manually move emails to or from spam. If this rises above 10 %, the model is drifting.

Step 7. Retrain. Re-run the pipeline on the latest 30 days of emails every week.

The pipeline is a cycle, not a one-time job. The model stays accurate because it keeps retraining on fresh data.

5 Three Generations of Machine Learning

ML has evolved through three distinct generations, each solving problems the previous generation could not.

Generation	Era	Key technique	What you need
Gen 1: Classic ML	1990s–2010s	Decision trees, SVMs, linear models	Labelled datasets, feature engineering
Gen 2: Deep Learning	2012–2020	Neural networks (CNN, RNN, LSTM)	Big data, GPU compute, labelled data
Gen 3: Transfer / Foundation	2020–now	Pre-trained models, fine-tuning, APIs	A task description; no training required

★ Everyday picture

Gen 1 is like hiring an apprentice who learned only what you explicitly taught them. Gen 2 is like hiring a savant who read millions of books — but you still had to supply the books. Gen 3 is like using a consultant who already knows everything. You tell them what you need and they deliver.

✓ Key takeaway

Gen 3 (Foundation Models) lowers the bar to entry. A developer adds ML to an app by calling an API. No training data needed.

6 Types of ML Domains

ML is applied across many domains. The three most prevalent in industry are:

6.1 Natural Language Processing (NLP)

NLP teaches machines to understand, generate, and reason about human language. Human language is ambiguous, context-dependent, and culturally loaded — making rule-based systems inadequate.

Examples: sentiment analysis, machine translation (Google Translate), document summarisation, question answering, chatbots.

6.2 Computer Vision (CV)

Computer Vision gives machines the ability to interpret images and video. Common uses: object detection (autonomous vehicles), face recognition (phone unlock), quality control (finding production-line defects), and medical imaging (detecting tumours).

6.3 Speech Recognition

Automatic Speech Recognition (ASR) converts spoken audio to text. Used in virtual assistants (Alexa, Siri, Google Assistant), transcription services, call-centre automation.

Where this shows up in ML/AI. Most modern ML apps combine domains. A voice assistant uses ASR (speech to text), then NLP (understand the question), then text-to-speech (answer). An autonomous car uses CV (see the road), NLP (process voice commands), and reinforcement learning (steer).

7 Foundation Models and Large Language Models

A **Foundation Model** is a large-scale model trained on a huge, broad dataset. Think: the entire web, all of Wikipedia, billions of images. After training it is adapted to specific tasks. Pre-training is expensive. Adaptation is cheap.

The intuition

Think of a foundation model as a university-educated generalist. They spent years learning widely (pre-training). You give them a two-week onboarding (fine-tuning) for your specific job. They are immediately productive. A Gen 1 specialist only knows what you explicitly taught them. They are only as good as the examples you gave them.

Large Language Models (LLMs) are the dominant type of foundation model today. They are trained on text. They can write, summarise, translate, answer questions, generate code, and reason step by step. Examples include GPT-4 (OpenAI), Claude 3.5 (Anthropic), Gemini (Google), LLaMA (Meta), DeepSeek, and Mistral.

7.1 What an AI Engineer Needs to Know

The modern **AI Engineer** role bridges software engineering and AI. Required skills:

- Software engineering fundamentals: Python, SQL, Git, CI/CD, REST APIs.
- LLM integration: prompt engineering, RAG (Retrieval-Augmented Generation), fine-tuning.
- Model Context Protocol (MCP): connecting LLMs to external tools.
- MLOps: versioning, monitoring, deployment pipelines.

8 Software Engineering vs Machine Learning Engineering

SE and ML engineering are siblings, not twins. Knowing their differences is the first step to using SE skills in ML projects.

Dimension	Traditional SE	ML Engineering
Specification	Clear, explicit (requirements doc)	Fuzzy — defined by data and metrics
Correctness	Deterministic: same input → same output	Probabilistic: output depends on model and data
Debugging	Trace code; add print statements	Inspect data; plot distributions; ablation studies
Testing	Unit tests with expected outputs	Test on held-out data; measure F1, AUC
Artefacts	Executable code	Executable code + trained model + dataset
Maintenance	Fix bugs, add features	Retrain; fix data pipelines; monitor drift
Failure mode	Exception thrown; stack trace	Silently wrong predictions

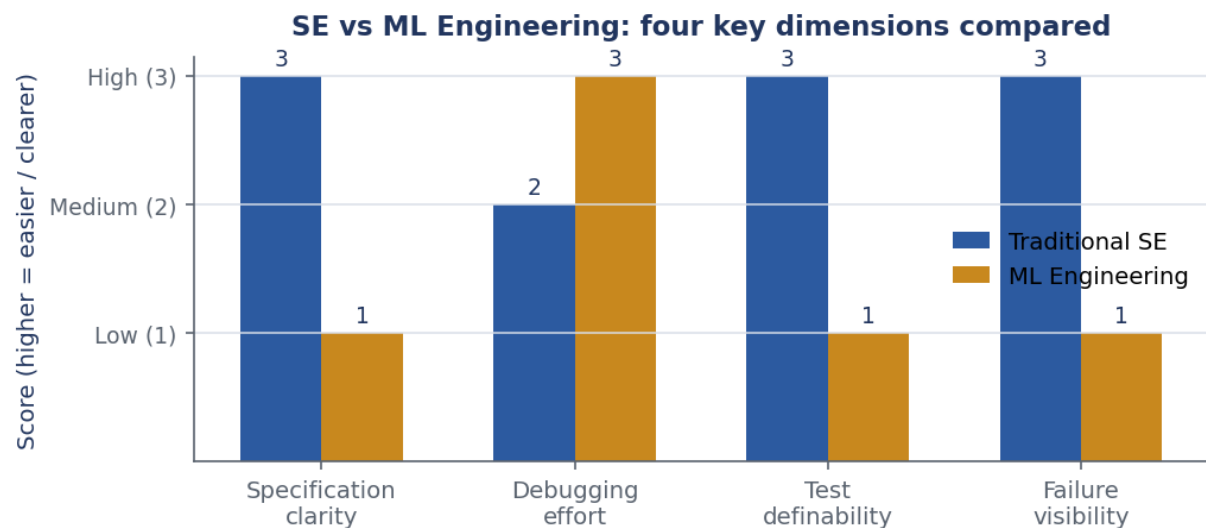


Figure 2: SE vs ML on four key dimensions. Blue bars show traditional SE; amber bars show ML engineering. Lower scores in ML mean harder and less predictable.

⚠ Watch out

The most dangerous ML failure is a *silent* one. A software bug crashes the program and announces itself. An ML model that starts giving wrong predictions may keep running for days before anyone notices. Monitoring is not optional.

📌 Worked example: SE rule vs ML model in a payment gateway

A payment gateway uses fraud detection. Compare what happens when a rule breaks vs when a model breaks.

- Step 1. SE rule breaks.** The rule “decline if country mismatches” fires incorrectly. The team adds one exception and redeploys in a few hours. Done.
- Step 2. ML model breaks.** The model starts declining legitimate transactions from a new market. It was never trained on data from that market.
- Step 3. The symptom is invisible.** Customer complaints rise slowly. No stack trace. No crash. The model keeps running and serving wrong answers.
- Step 4. Fixing the ML model takes days.** Inspect model predictions on the new market. Check if the training set included that market. Retrain the model. Evaluate on a fresh test set. Then redeploy.
- Step 5. So what?** SE skills help you catch this early. Set up monitoring. Write tests for model behaviour. Version the training data.

This is why adapting SE skills to ML is the whole point of this course.

✓ **Key takeaway**

ML shifts the source of truth from code to data. That one fact changes how you specify, test, debug, and maintain a system.

Glossary & Symbols

Key terms.

SDLC	Software Development Life Cycle. A structured sequence of phases: plan, design, build, test, deploy, maintain.
Agile	A way of working in short sprints (1–3 weeks). Teams deliver working software continuously.
DevOps	A practice where development and operations share responsibility. Testing and deployment are automated.
Cloud Native	Building systems using containers, microservices, and cloud infrastructure. Systems are small and independent.
Data Science	The study of data: collecting it, cleaning it, analysing it, and drawing useful insights from it.

Machine Learning

A branch of AI where a program learns from data instead of being hand-programmed with rules.

ML Pipeline	The end-to-end sequence from raw data to a deployed, monitored model.
Model	A trained algorithm that takes inputs and produces predictions or decisions.
Data Drift	A change in the distribution of real-world data that makes an old model less accurate.

Foundation Model

A large model pre-trained on broad data, then adapted to specific tasks.

LLM	Large Language Model. A foundation model trained on text; can write, summarise, translate, and answer questions.
Fine-tuning	Extra training on task-specific data to adapt a foundation model.
RAG	Retrieval-Augmented Generation. A method that gives an LLM access to a document store at query time.
MLOps	Practices and tools for versioning, deploying, monitoring, and retraining ML models in production.
NLP	Natural Language Processing. Teaching machines to understand and generate human language.

ASR	Automatic Speech Recognition. Converts spoken audio to text.
CV	Computer Vision. Gives machines the ability to interpret images and video.

Abbreviations at a glance.

Abbreviation	Stands for
SDLC	Software Development Life Cycle
ML	Machine Learning
AI	Artificial Intelligence
LLM	Large Language Model
NLP	Natural Language Processing
ASR	Automatic Speech Recognition
CV	Computer Vision
RAG	Retrieval-Augmented Generation
MLOps	Machine Learning Operations
MCP	Model Context Protocol
CI/CD	Continuous Integration / Continuous Deployment
AUC	Area Under the ROC Curve (a model quality measure)